

AFAE - Increasing the accuracy

Delmotte Aurélien

Zhang Xiru

October 2024

Contents

1	Introduction	2
2	Polynomial Evaluation	2
2.1	Classic Horner's Scheme	2
2.2	The Error-Free Transformations (EFT)	2
2.3	Compensated Horner's Scheme	3
2.4	Horner Scheme in Exact Arithmetic	3
2.5	Computing the Condition Number	3
2.6	Performance Evaluation	3
3	Summation Algorithms	4
3.1	Classic Recursive Summation	4
3.2	Kahan's Summation	4
3.3	Priest's Doubly Compensated Summation	5
3.4	Rump's Compensated Summation	5
3.5	Performance Evaluation	6

1 Introduction

Addressing the challenges of floating-point arithmetic, particularly in polynomial evaluations and summation algorithms, is critical in numerical computations. This study implements and evaluates advanced algorithms that aim to minimize rounding errors and improve accuracy, providing insights into their efficacy across varying conditions.

2 Polynomial Evaluation

2.1 Classic Horner's Scheme

To evaluate the polynomial $p(x) = \sum_{i=0}^n a_i x^i$ at a given point x , the classical Horner's algorithm starts with the highest degree coefficient of the polynomial and accumulates each coefficient, effectively reducing the number of operations in the computation. Below is the implementation 1.

Listing 1: Classic Horner Function

```
1 function res = ClassicHorner(p,x)
2     p = double(p);
3     x = double(x);
4
5     res = double(0);
6     for i = 1 : length(p)
7         res = double(res*x);
8         res = double(res+p(i));
9     end
10 end
```

The implementation of Horner's method was validated by comparing its results with those from MATLAB's `polyval`. Consistent matches across various test cases confirm the accuracy of our implementation.

2.2 The Error-Free Transformations (EFT)

The following MATLAB functions implement various Error-Free Transformations (EFTs) that enhance the accuracy of basic arithmetic operations, such as addition, subtraction, and multiplication, on floating-point numbers.

Listing 2: Error-Free Transformations (EFT)

```
1 % EFT for the summation of two floating-point numbers with |a| >= |b|
2 function [x,y] = FastTwoSum(a,b)
3     a = double(a);
4     b = double(b);
5
6     x = double(a+b);
7     y = double((a-x)+b);
8 end
9 % EFT for the summation of two floating-point numbers
10 function [x,y] = TwoSum(a,b)
11     a = double(a);
12     b = double(b);
13
14     x = double(a+b);
15     z = double(x-a);
16     y = double((a-(x-z))+(b-z));
17 end
18 % Error-free split of a floating point number into two parts
19 function [x,y] = Split(a)
20     a = double(a);
21
22     factor = double(2^27+1); %27=s
23     c = double(factor*a);
24     x = double(c-(c-a));
25     y = double(a-x);
26 end
27 % EFT of the product of two floating-point numbers
28 function [x,y] = TwoProduct(a,b)
29     a = double(a);
30     b = double(b);
31
32     x = double(a*b);
33     [a_1,a_2] = Split(a);
34     [b_1,b_2] = Split(b);
35     y = double(a_2*b_2-(((x-a_1*b_1)-a_2*b_1)-a_1*b_2));
36 end
```

2.3 Compensated Horner's Scheme

The Compensated Horner's Scheme enhances the classic Horner's method by incorporating error-free transformations, thus ensuring higher accuracy in evaluating polynomials. The implementation is presented below.

Listing 3: Compensated Horner's Scheme

```

1 function res = CompensatedHorner(p,x)
2   p = double(flip(p));
3   x = double(x);
4
5   n = length(p);
6   s = double(zeros(1,length(p)));
7   s(n) = p(n);
8   r = double(zeros(1,length(p)));
9   for i = n-1:-1:1
10      [p_i,pi_i] = TwoProduct(s(i+1),x);
11      [s(i),sig_i] = TwoSum(p_i,p(i));
12      r(i) = double(r(i+1)*x+(pi_i+sig_i));
13   end
14   res = s(1)+r(1);
15 end

```

2.4 Horner Scheme in Exact Arithmetic

To avoid the inaccuracies associated with floating-point arithmetic, the Horner scheme can also be implemented using symbolic calculations. This approach ensures that the polynomial evaluation is performed with exact arithmetic, preserving all mathematical properties without approximation. Below is the implementation using the Symbolic Math Toolbox.

Listing 4: Horner Scheme in Exact Arithmetic

```

1 function res = SymSolve(p,a)
2   syms f(x)
3   f(x) = poly2sym(p);
4   res = f(a);
5 end
6
7 function res = SymHorner(p,x)
8   syms pp xx
9   pp = p;
10  xx = x;
11  res = double(0);
12  for i = 1:length(p)
13     res = double(res*xx);
14     res = double(res+pp(i));
15  end
16 end

```

2.5 Computing the Condition Number

The condition number of a polynomial at a given point provides a measure of sensitivity of the polynomial's value to changes in the input value. It is computed as the ratio of the relative change in the polynomial's output to the relative change in the input. This can be crucial for understanding the stability of numerical computations involving polynomials.

Listing 5: Condition Number

```

1 function res = condp(p,x)
2   res = SymSolve(abs(p),abs(x))/abs(SymSolve(p,x));
3 end

```

2.6 Performance Evaluation

Tests were conducted on the polynomial $p_n(x) = (x - 1)^n$ with $x = fl(1.333)$ and degrees ranging from 3 to 42 to assess the accuracy of the classic and compensated Horner schemes. The following figure illustrates the relative errors and condition numbers.

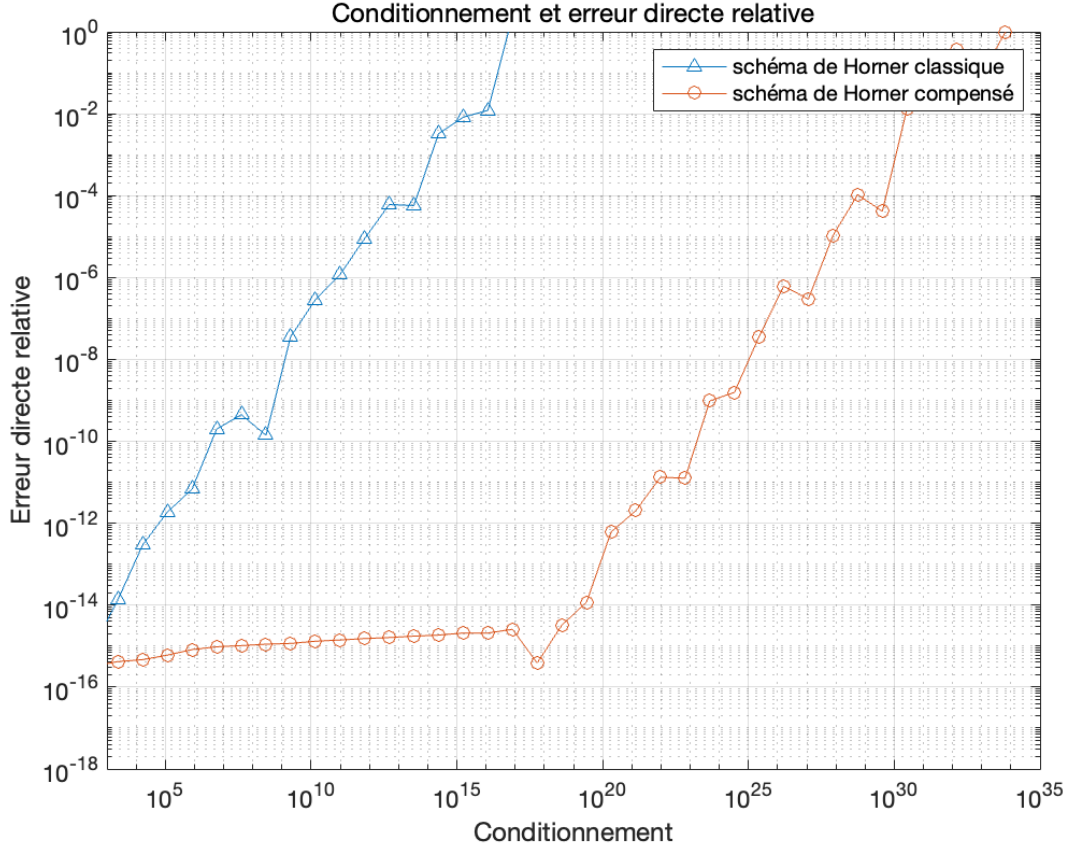


Figure 1: Relative errors and condition numbers for different polynomial degrees evaluated using classic and compensated Horner schemes.

We do indeed notice that we stay accurate for much larger condition numbers using the compensated Horner scheme, as would be expected from theory.

3 Summation Algorithms

3.1 Classic Recursive Summation

The "naive" approach to summation consists of adding all the numbers one by one to an accumulator initialized at zero. The algorithm is as follows:

Listing 6: Classic Recursive Summation

```

1 function res = Sum(p)
2     sig = 0;
3     for i = 1:length(p)
4         sig = double(sig + p(i));
5     end
6     res = sig;
7 end

```

The relative error for this algorithm is approximately $\gamma_{n-1} \times \text{cond}(p)$, where γ_n is the accumulation of rounding errors over n operations, and $\text{cond}(p)$ is the condition number of the summation.

3.2 Kahan's Summation

Kahan's summation algorithm improves accuracy by compensating for the floating-point error at each step. The error from the previous addition is carried forward and incorporated into the next step, leading to the following implementation:

Listing 7: Kahan's Summation

```

1 function res = SCompSum(p)
2     sig = 0;
3     e = 0;
4     for i = 1:length(p)
5         y = double(p(i) + e);
6         [sig, e] = FastTwoSum(sig, y);
7     end
8     res = sig;
9 end

```

For Kahan's summation, the relative error is bounded by $\mathbf{u} + \mathcal{O}(n\mathbf{u}^2) \times \text{cond}(p)$, where $\mathbf{u}$ is the unit round-off error. This provides significant improvement over the classic summation method.

3.3 Priest's Doubly Compensated Summation

Priest's algorithm enhances accuracy by introducing an additional layer of error compensation and sorting the input array:

Listing 8: Priest's Doubly Compensated Summation

```

1 function res = DCompSum(p)
2     ps = sort(p); % Sorting step can be omitted if not needed
3     s = 0;
4     c = 0;
5     for i = 1:length(p)
6         [y, u] = FastTwoSum(c, ps(i));
7         [t, v] = FastTwoSum(s, y);
8         z = double(u + v);
9         [s, c] = FastTwoSum(t, z);
10    end
11    res = s;
12 end

```

This algorithm achieves nearly perfect relative accuracy of $2\mathbf{u}$, losing only a single bit of precision, regardless of the condition number. However, if sorting is used, it introduces an $\mathcal{O}(n \log n)$ time complexity, which may slow the algorithm down.

3.4 Rump's Compensated Summation

Rump's compensated summation strikes a balance between speed and accuracy by maintaining a separate accumulator for errors. Instead of adjusting the next element with the current error, the errors are accumulated and incorporated into the final result:

Listing 9: Rump's Compensated Summation

```

1 function res = CompSum(p)
2     n = length(p);
3     pi = zeros(n);
4     sig = zeros(n);
5     pi(1) = p(1);
6     sig(1) = 0;
7     for i = 2:n
8         [pi(i), qi] = TwoSum(pi(i-1), p(i));
9         sig(i) = double(sig(i-1) + qi);
10    end
11    res = double(pi(n) + sig(n));
12 end

```

For Rump's algorithm, the relative error is approximately $\mathcal{O}(n\mathbf{u}^2) \times \text{cond}(p)$, offering excellent accuracy for well-conditioned sums while still maintaining good performance.

3.5 Performance Evaluation

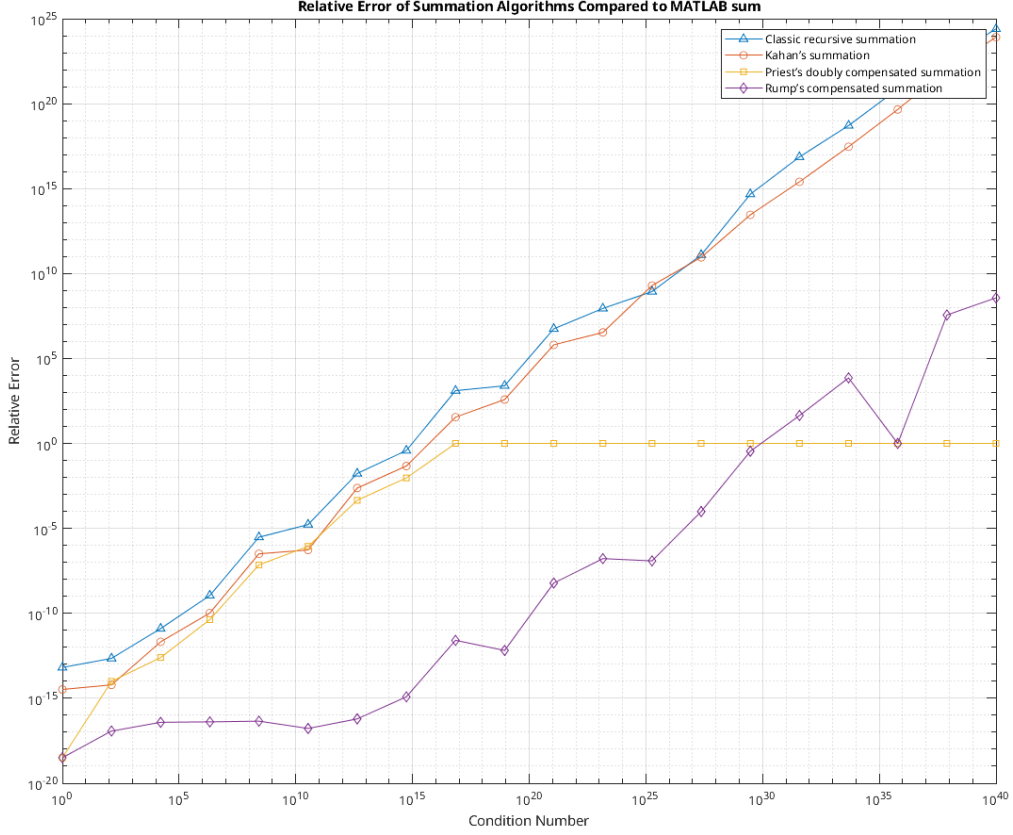


Figure 2: Relative errors against condition numbers for the Sum of different arrays evaluated using different algorithms.

As expected from theoretical analysis, the classic recursive summation yields the worst accuracy. Kahan's summation shows a modest improvement. Priest's algorithm performs best for larger condition numbers, as its accuracy is independent of the condition number. Rump's summation, on the other hand, is optimal for lower condition numbers, only losing accuracy once the condition number exceeds $1/\mathbf{u}$. However, unlike Priest's method, Rump's algorithm does not flatten out at higher condition numbers as it starts to lose stability for higher condition numbers.

All in all Rump's algorithm is the most accurate practical (as in $\mathcal{O}(n)$) algorithm tested.